

# Bachelor AI - MLOps

20 FEATURE STORE

Tobias Mérinat

`tobias.merinat@hslu.ch`



DEPARTMENT OF INFORMATION TECHNOLOGY  
Lucerne University of Applied Sciences and Arts  
6343 Rotkreuz, Switzerland

September 17/24 and October 15 2026

# Recap: Training-Serving Skew and Data Leakage

## Reasons for Training Serving Skew:

- Non-representative training data
- Incorrect Cross Validation (time series, normalization across validation data, grouped data)
- Data drift
- Errors in reimplementing feature pipelines
- Errors in joining features

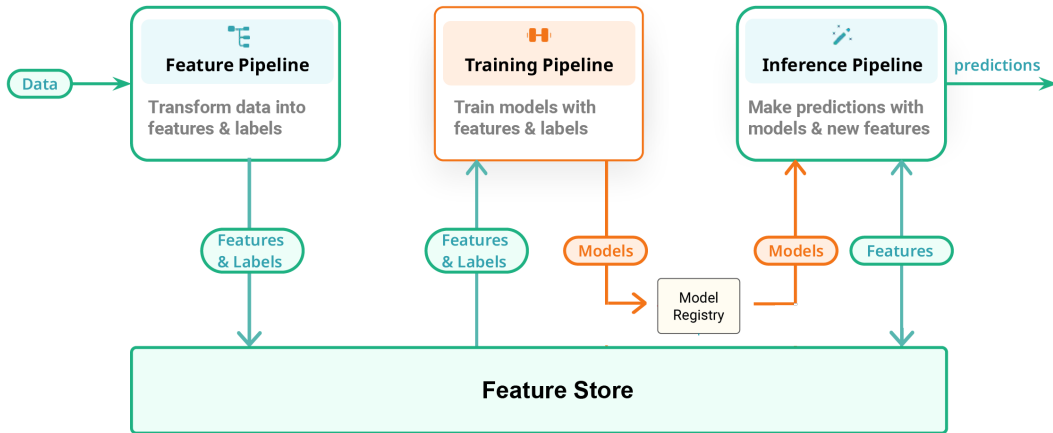
Three main tasks:

- 1 Persist features and labels to provide historical data for training
- 2 Reduce latency during online inference
- 3 Provide a catalog to make features searchable

A Feature Store must therefore

- Receive features via batch and streaming APIs
- Store features efficiently
- Enable feature management and discovery
- Provide features for batch and online inference
- Provide features for training

# Architektur mit Feature Store (hopsworks)



For use during inference, a Feature Store offers two (or three) access methods or databases:

- an online store with low latency (online prediction)
- an offline store for large amounts of historical data (batch prediction)
- optionally a vector database for storing and finding embeddings

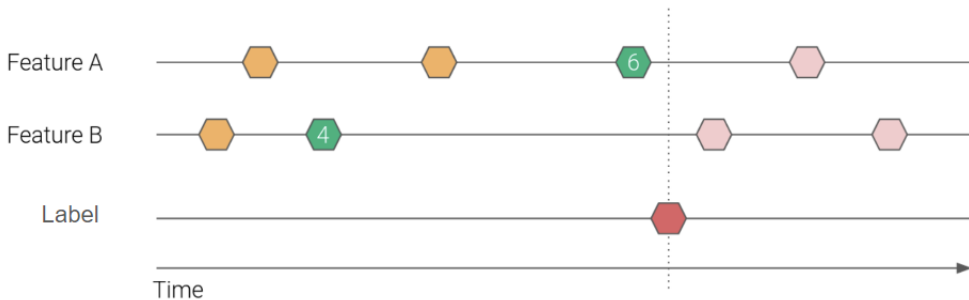
- Backfilling and Point-in-Time Correctness
- Feature catalog with discovery
- In general: Central repository for features with APIs for writing, managing and using

- The process of computing a (feature) dataset from historical raw data
- That is, running a feature pipeline with *from/to* specifications to generate features
- Both batch and streaming pipelines should support backfilling
- Necessary to generate training data (for retraining or new models)
- Necessary when a new feature is developed or an existing one is changed
- The *same* feature pipeline should be used for *live* data and for backfilling
- Without backfilling, the *log and wait* approach is necessary



- **Point-in-time Correctness:** The ability of a system to perform a calculation exactly as it would have occurred at any point in the past.
- **Point-in-time Correct Join:** A temporal join between tables whose result reflects the state of both tables at a given point in time. The left table with the labels determines the point in time.
- With point-in-time correct joins, no data is leaked when merging tables.

## Point-in-Time Correct Training Data



# Point-in-time Beispiel

**seller\_delivery\_time\_monthly**

| seller_id | event_time | avg_deliver_time_hrs |
|-----------|------------|----------------------|
| 1112      | oct-22     | 72                   |
| 1112      | nov-22     | 104                  |
| 1112      | dec-22     | 88                   |

**seller\_reviews\_quarterly**

| seller_id | event_time | avg_review_score | ... |
|-----------|------------|------------------|-----|
| 1112      | oct-22     | 3.4              | ... |
| 1112      | dec-22     | 2.8              | ... |

seller\_delivery\_time\_monthly is the label Feature Group. Join on seller\_id, lining up rows on event\_time (point-in-time join).

**seller\_delivery\_time\_monthly\_fv**

| seller_id | event_time | avg_deliver_time_hrs | avg_review_score | ... |
|-----------|------------|----------------------|------------------|-----|
|-----------|------------|----------------------|------------------|-----|

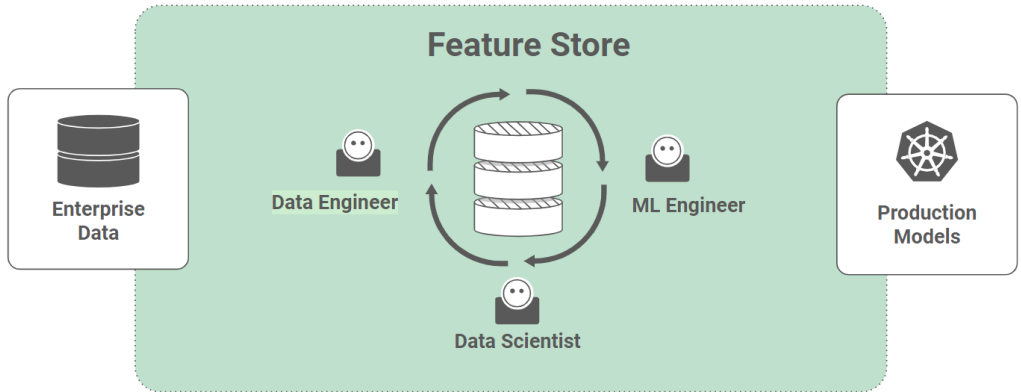
*seller\_delivery\_time\_monthly\_fv.  
save\_training\_data(..,format="csv",..)*

**seller\_delivery\_time\_monthly\_td.csv**

| seller_id | event_time | avg_deliver_time_hrs | avg_review_score | ... |
|-----------|------------|----------------------|------------------|-----|
| 1112      | oct-22     | 72                   | 3.4              |     |
| 1112      | nov-22     | 104                  | 3.4              |     |
| 1112      | dec-22     | 88                   | 2.8              |     |

Point-in-Time  
correct JOIN

## Cross-Team Collaboration with a Feature Store



# Summary of Feature Store Tasks

- Feature Repository (Offline Store):
  - API for using features (insert, validation, inference, training)
- Online Store: Enables use cases where inference latency must be low
- Feature Catalog:
  - Central repository for features, search, supports reusability
  - Provides standards for naming features, metadata and documentation
- Point-in-time Correctness:
  - Creating training sets becomes simpler and less error-prone
  - Faster iterations through automated backfilling (vs. log-and-wait approach)
- Data Governance:
  - Feature lineage and backfilling helps with reproducibility
  - Access restrictions (role-based access, automated masking/anonymization, propagating restrictions to derived features)

- Hopsworks Featurestore
- Feast
- Redis Feature Form
- Large list and comparisons on [featurestore.org](https://featurestore.org)